

GPUMODE Lecture Study Notes: Scaling Laws for Low Precision

Core Problem the Lecture Addresses

This lecture studies how numerical precision affects language model loss during post-training quantization, quantization-aware training, and low-precision training. The central question is whether the loss penalty from low precision can be predicted using scaling-law-style reasoning.

The motivating question is:

Given a fixed compute or serving budget, is it better to use a larger model at lower precision or a smaller model at higher precision?

For example, compare:

- a 1 billion parameter model whose weights are quantized to 4 bits;
- a 500 million parameter model kept in high precision, such as BF16 or FP16.

It is not obvious which model should perform better. A larger model has more capacity, but low precision injects numerical error. A smaller model has less capacity, but its computations are more faithful to the trained parameters.

The lecture frames quantization as a perturbation:

$$q(x) = x + \epsilon,$$

where $q(x)$ is the quantized and dequantized value, and ϵ is the quantization error. If the statistics of ϵ are predictable as a function of bitwidth, then the loss increase may also be predictable.

The key quantities are:

- N : number of model parameters, usually non-embedding parameters;
- D : number of training tokens;
- p : precision in bits;

- $L(N, D, p)$: model loss as a function of size, data, and precision;
- ΔL : loss degradation caused by quantization.

The lecture’s main message is that precision is not just an implementation detail. It is a scaling variable, similar to model size and data.

Required Background

Classical Neural Scaling Laws

A simplified neural scaling law expresses loss as:

$$L(N, D) \approx L_\infty + AN^{-\alpha} + BD^{-\beta},$$

where:

- L_∞ is the irreducible loss floor;
- $AN^{-\alpha}$ is the loss contribution from finite model size;
- $BD^{-\beta}$ is the loss contribution from finite data;
- α and β are empirical scaling exponents.

The Chinchilla-style perspective says that for compute-optimal training, model size and data should be balanced. A common approximate rule is:

$$D \approx 20N.$$

A model trained with much more data than this is called **overtrained**. Modern production models are often deliberately overtrained because inference cost matters. A smaller model trained longer may be cheaper to serve than a larger model trained near the classical compute-optimal point.

Quantization

Quantization maps high-precision values to a smaller finite set of representable values. A simple symmetric integer quantizer is:

$$q(x) = s \cdot \text{clip} \left(\text{round} \left(\frac{x}{s} \right), -q_{\max}, q_{\max} \right),$$

where:

- x is a high-precision scalar;
- s is the quantization scale;
- $q_{\max}$ is the largest representable integer magnitude;

- $q(x)$ is the dequantized low-precision approximation.

The quantization error is:

$$\epsilon(x) = q(x) - x.$$

For a uniform quantizer with step size Δ , a simple approximation is:

$$\epsilon \sim \text{Uniform}\left(-\frac{\Delta}{2}, \frac{\Delta}{2}\right), \quad \text{Var}(\epsilon) = \frac{\Delta^2}{12}.$$

This is only a first-order mental model. Modern methods such as GPTQ and AWQ do not simply add independent uniform noise, but the noise view is still useful for understanding why lower precision should increase loss.

Post-Training Quantization

In post-training quantization, the model is first trained normally and then quantized afterward:

high-precision training $\rightarrow$ weight quantization $\rightarrow$ low-precision inference.

The training process does not adapt to quantization. Therefore, post-training quantization measures how robust the trained high-precision checkpoint is to perturbation.

Quantization-Aware Training

In quantization-aware training, the forward pass simulates quantization during training. A simplified linear layer becomes:

$$y = xq(W)^T,$$

where $q(W)$ is the quantized version of the weight matrix. The optimizer may still maintain high-precision master weights.

The purpose is to let training adapt to the quantization constraint.

Low-Precision Training

In true low-precision training, the actual matrix multiplications are performed in low precision. This can require low-precision versions of:

- weights;

- activations;
- gradients;
- optimizer states;
- key-value cache tensors.

Modern GPUs expose different tensor-core throughput depending on dtype. For example, FP8 or FP4 matrix multiplication can have much higher theoretical throughput than FP16 or BF16, but only if the hardware and kernels actually support the relevant computation efficiently.

Main Concepts

Why BitNet and Aggressive Quantization Motivated the Work

The lecture was partly motivated by very low-precision models such as BitNet-style systems. These works ask whether language models can be trained with extremely few states per weight, such as binary or ternary weights.

However, the lecture does not claim to reproduce the full BitNet setup. Instead, it studies a controlled baseline:

- use a standard causal transformer;
- vary numerical precision;
- measure the resulting loss;
- fit scaling-law-style functional forms.

This matters because specialized low-precision architectures may change many things at once: normalization, activation functions, optimizer schedules, learning rates, and weight decay. The lecture's goal is to isolate the effect of precision itself.

The Scaling Culture Versus the Systems Culture

A recurring theme is the tension between two styles of thinking.

The scaling-law view asks:

What is the functional form of $L(N, D, p)$?

It cares about smooth trends, asymptotics, and fitted exponents.

The systems view asks:

- Is quantization per tensor, per channel, or per group?

- Is the format integer or floating point?
- Are values packed into memory?
- Does the GPU have a native tensor-core path?
- Is dequantization fused into the matrix multiplication kernel?
- Are memory accesses coalesced?
- Are the scale factors cached efficiently?

The lecture attempts to connect both views. The scaling-law fit abstracts over implementation details, but the experiments still check whether the broad trends are robust to common quantization methods.

Main Post-Training Quantization Result

The first major result is:

Overtrained models are hurt more by post-training quantization.

Let:

$$\Delta L_{\text{PTQ}}(N, D, p) = L_{\text{quantized}}(N, D, p) - L_{\text{full}}(N, D).$$

The empirical finding is that ΔL_{PTQ} increases predictably with the token-to-parameter ratio D/N .

A generic fitted form is:

$$\Delta L_{\text{PTQ}}(N, D, p) \approx A(p) \left(\frac{D}{N} \right)^\gamma,$$

where:

- $A(p)$ increases as bitwidth decreases;
- $\gamma > 0$ measures how sensitivity grows with overtraining;
- D/N measures how many tokens each parameter has absorbed.

The important qualitative derivative is:

$$\frac{\partial \Delta L_{\text{PTQ}}}{\partial (D/N)} > 0.$$

Thus, even if full-precision loss improves with more data, quantization sensitivity also increases.

Why More Pre-Training Compute Can Hurt the Served Model

The served model after quantization has loss:

$$L_{\text{serve}}(N, D, p) = L_{\text{full}}(N, D) + \Delta L_{\text{PTQ}}(N, D, p).$$

Using a simple scaling model:

$$L_{\text{full}}(N, D) = L_{\infty} + AN^{-\alpha} + BD^{-\beta},$$

and a quantization penalty:

$$\Delta L_{\text{PTQ}}(N, D, p) = Q(p) \left(\frac{D}{N} \right)^{\gamma},$$

we get:

$$L_{\text{serve}}(N, D, p) = L_{\infty} + AN^{-\alpha} + BD^{-\beta} + Q(p) \left(\frac{D}{N} \right)^{\gamma}.$$

The term $BD^{-\beta}$ decreases with data. The term $Q(p)(D/N)^{\gamma}$ increases with data per parameter. Therefore, at sufficiently low precision, the served loss can become U-shaped as a function of training tokens.

This is the surprising conclusion:

More pre-training can improve the high-precision checkpoint while worsening the quantized checkpoint.

Critical Training Point Under Post-Training Quantization

The optimum post-quantization data budget occurs when the marginal benefit of more data equals the marginal quantization damage.

Differentiate:

$$\frac{\partial L_{\text{serve}}}{\partial D} = -\beta BD^{-\beta-1} + Q(p)\gamma \frac{1}{N} \left(\frac{D}{N} \right)^{\gamma-1}.$$

At the optimum:

$$\beta BD^{-\beta-1} = Q(p)\gamma \frac{1}{N} \left(\frac{D}{N} \right)^{\gamma-1}.$$

This equation says:

Stop training, for a fixed quantized serving format, when the full-precision gain from extra data is exactly balanced by the added fragility to quantization.

For 8-bit quantization, the critical point may be far beyond practical token budgets. For 3-bit or 4-bit quantization, the critical point can become practically relevant.

Interpretation of the U-Shaped Curve

The U-shape has three regimes:

1. **Undertrained regime:** more data helps strongly, and quantization damage is small.
2. **Transition regime:** more data helps, but the gain is partially offset by increasing quantization sensitivity.
3. **Overtrained low-precision regime:** more data barely improves full-precision loss but significantly increases quantization damage.

The practical lesson is:

The best full-precision checkpoint is not necessarily the best checkpoint for low-precision deployment.

Connection to Llama 2 and Llama 3

The lecture discusses the observation that Llama 3 appeared harder to quantize naively than Llama 2. A plausible explanation is that Llama 3 was trained on far more tokens per parameter.

The proposed mental model is:

more overtraining $\Rightarrow$ more information packed into each parameter $\Rightarrow$ greater sensitivity to perturbation

This does not mean Llama 3 cannot be quantized. It means naive post-training quantization may need more careful techniques, such as avoiding sensitive layers, using better calibration, or choosing more advanced quantization schemes.

Quantization During Training

The lecture then moves from post-training quantization to quantization during training. The goal is to predict:

$$L(N, D, p_W, p_A, p_{KV}),$$

where:

- p_W is weight precision;
- p_A is activation precision;
- p_{KV} is key-value cache precision.

The important point is that different parts of the model have different sensitivity to quantization.

A useful decomposition is:

$$L(N, D, p_W, p_A, p_{KV}) \approx L_{\text{base}}(N, D) + \Delta L_W(p_W) + \Delta L_A(p_A) + \Delta L_{KV}(p_{KV}).$$

This is not exact, but it is a useful modeling structure.

Effective Parameter Count

The lecture introduces the idea that low precision can be interpreted as reducing the model's effective parameter count.

Suppose a full-precision model with N_{eff} parameters has the same loss as a quantized model with N actual parameters. Then:

$$L_{\text{quant}}(N, p) \approx L_{\text{full}}(N_{\text{eff}}).$$

Define:

$$N_{\text{eff}}(p) = r(p)N,$$

where $r(p)$ is the effective parameter multiplier.

Examples:

- If $r(p) = 1$, quantization has no loss-equivalent capacity penalty.
- If $r(p) = 0.8$, quantization behaves like removing 20% of the useful parameters.
- If $r(p) = 0.5$, quantization behaves like cutting the model's useful capacity in half.

This gives an intuitive way to compare precision and scale.

Nonlinear Trade-Off Between Precision and Scale

A key result is that the trade-off between bitwidth and parameter count is smooth but strongly nonlinear. The reason is intuitive: p bits can represent 2^p states.

Thus, the damage from reducing precision is not linear in p . Going from 16 bits to 8 bits may be mild, but going from 5 bits to 4 bits or from 4 bits to 3 bits can be severe.

The isoloss relationship is:

$$L(N, p) = \ell.$$

Along an isoloss contour, if precision decreases, parameter count must increase:

$$p \downarrow \Rightarrow N \uparrow.$$

At low precision, the required increase in N can become very large.

Hardware-Level Explanation

Why Low Precision Is Attractive on GPUs

Low precision helps GPUs in two main ways:

- It reduces memory traffic.
- It can increase tensor-core throughput.

A tensor with M elements at p bits requires:

$$\text{Memory} = \frac{Mp}{8} \text{ bytes.}$$

Reducing from 16 bits to 8 bits halves memory:

$$\frac{8}{16} = \frac{1}{2}.$$

Reducing from 8 bits to 4 bits halves memory again:

$$\frac{4}{8} = \frac{1}{2}.$$

For a model with N parameters, weight memory is:

$$M_W = \frac{Np_W}{8}.$$

For a 70 billion parameter model:

$$M_W(p_W = 16) \approx 140 \text{ GB,}$$

$$M_W(p_W = 8) \approx 70 \text{ GB},$$

$$M_W(p_W = 4) \approx 35 \text{ GB}.$$

Thus, quantization can change whether a model fits on one GPU, multiple GPUs, or not at all.

Weights

Weights are fixed tensors after training. They are often easier to quantize than activations because calibration can be performed once.

For a linear layer:

$$Y = XW^T,$$

weight-only quantization replaces W with $\widehat{W}$:

$$Y = X\widehat{W}^T.$$

If:

$$\widehat{W} = W + E_W,$$

then:

$$Y_{\text{quant}} = X(W + E_W)^T = XW^T + XE_W^T.$$

The output error is:

$$\Delta Y = XE_W^T.$$

The effect depends on both the quantization error and the input activations.

Activations

Activations are input-dependent and often have outliers. Their memory during training roughly scales as:

$$M_A \propto BSLH \frac{p_A}{8},$$

where:

- B is batch size;
- S is sequence length;
- L is number of layers;
- H is hidden dimension;
- p_A is activation precision.

Activations are often more sensitive to quantization because their distribution varies across batches, layers, and token positions.

Key-Value Cache

During autoregressive decoding, the KV cache stores previous keys and values. Its memory is approximately:

$$M_{KV} = 2BSLH_{kv} \frac{p_{KV}}{8}.$$

The factor 2 comes from storing both keys and values.

For long-context inference, S can be large, making the KV cache a major memory bottleneck. Quantizing it can greatly improve serving throughput and memory capacity.

Attention scores are:

$$S_{\text{attn}} = \frac{QK^T}{\sqrt{d_k}}.$$

If keys are quantized:

$$\hat{K} = K + E_K,$$

then:

$$\hat{S}_{\text{attn}} = \frac{Q\hat{K}^T}{\sqrt{d_k}} = \frac{QK^T}{\sqrt{d_k}} + \frac{QE_K^T}{\sqrt{d_k}}.$$

The score perturbation is:

$$\Delta S_{\text{attn}} = \frac{QE_K^T}{\sqrt{d_k}}.$$

Because attention uses a softmax, small logit changes can change the output distribution. If values are also quantized:

$$\hat{V} = V + E_V,$$

then the attention output becomes:

$$\hat{O} = \text{softmax}(\hat{S}_{\text{attn}})\hat{V}.$$

Thus KV cache quantization affects both where the model attends and what information it retrieves.

Arithmetic Intensity

Arithmetic intensity is:

$$\text{AI} = \frac{\text{FLOPs}}{\text{bytes moved}}.$$

A kernel is memory-bound if:

$$\text{AI} < \frac{\text{peak FLOPs}}{\text{peak bandwidth}}.$$

It is compute-bound if:

$$\text{AI} > \frac{\text{peak FLOPs}}{\text{peak bandwidth}}.$$

Low precision improves arithmetic intensity by reducing bytes moved. If the same operation moves half as many bytes, then arithmetic intensity doubles:

$$\text{AI}_8 \approx 2 \text{AI}_{16}.$$

For 4 bits relative to 16 bits:

$$\text{AI}_4 \approx 4 \text{AI}_{16}.$$

This is why low precision can be especially powerful for memory-bound inference workloads.

Matrix Multiplication Throughput

A dense matrix multiplication:

$$C = AB,$$

with:

$$A \in \mathbb{R}^{M \times K}, \quad B \in \mathbb{R}^{K \times N}, \quad C \in \mathbb{R}^{M \times N},$$

requires approximately:

$$2MKN$$

floating-point operations.

On tensor-core GPUs, lower precision can increase peak throughput. However, actual speed depends on whether the implementation avoids overhead from:

- loading scale factors;
- unpacking low-bit values;
- dequantizing into registers;
- handling irregular memory layouts;
- synchronizing within thread blocks;
- and failing to use tensor-core-friendly tile shapes.

Visual Concept

Draw global memory feeding quantized weights, activations, and KV cache into an SM. Inside the SM, draw shared memory, registers, tensor cores, and warp schedulers. Label where low precision reduces bandwidth and where tensor cores can increase throughput.

Mathematical Reasoning

Extended Scaling Law With Precision

A natural precision-aware scaling law is:

$$L(N, D, p) = L_{\infty} + AN^{-\alpha} + BD^{-\beta} + R(p, N, D),$$

where $R(p, N, D)$ is the precision penalty.

For multiple tensor groups:

$$L(N, D, p_W, p_A, p_{KV}) = L_\infty + AN^{-\alpha} + BD^{-\beta} + R_W(p_W) + R_A(p_A) + R_{KV}(p_{KV}).$$

This compositional structure is useful because it lets us reason about which tensor group is most fragile.

Power Law for Post-Training Quantization Damage

The post-training quantization result can be summarized as:

$$\Delta L_{\text{PTQ}} = C_p \left(\frac{D}{N} \right)^\gamma.$$

Taking logs:

$$\log \Delta L_{\text{PTQ}} = \log C_p + \gamma \log \left(\frac{D}{N} \right).$$

Thus, if a log-log plot is approximately linear, the degradation follows a power law.

The bitwidth dependence is contained in C_p . Lower precision means larger C_p :

$$p_1 < p_2 \quad \Rightarrow \quad C_{p_1} > C_{p_2}.$$

Effective Parameter Count Derivation

Suppose full-precision loss as a function of parameter count is:

$$L_{\text{full}}(N) = L_\infty + AN^{-\alpha}.$$

Given a quantized model with loss L_{quant} , define N_{eff} by:

$$L_{\text{quant}} = L_\infty + AN_{\text{eff}}^{-\alpha}.$$

Then:

$$N_{\text{eff}}^{-\alpha} = \frac{L_{\text{quant}} - L_\infty}{A}.$$

Solving:

$$N_{\text{eff}} = \left(\frac{A}{L_{\text{quant}} - L_{\infty}} \right)^{1/\alpha}.$$

The effective parameter multiplier is:

$$r(p) = \frac{N_{\text{eff}}(p)}{N}.$$

This converts quantization damage into a model-size-equivalent penalty.

Compute-Optimal Precision

Assume training compute is approximately:

$$C \approx kND.$$

If precision changes the cost per operation, write:

$$C \approx k(p)ND.$$

The optimization problem for fixed parameter count is:

$$\min_{D,p} L(N, D, p)$$

subject to:

$$C = k(p)ND.$$

The lecture reports that the optimal precision grows approximately logarithmically with compute:

$$p^* \propto \log C.$$

The intuition is that the statistical penalty of too-low precision falls exponentially with bitwidth, while compute benefits from lower precision are closer to linear or piecewise linear in practical hardware regimes.

Toy Derivation of Logarithmic Optimal Precision

Suppose the precision penalty is:

$$R(p) = Qe^{-\lambda p}.$$

Suppose lower precision gives more effective compute, and the compute-related loss term is:

$$L_{\text{compute}}(p) = A \left(\frac{C}{p} \right)^{-\alpha} = AC^{-\alpha} p^{\alpha}.$$

Then:

$$L(p) = AC^{-\alpha} p^{\alpha} + Qe^{-\lambda p}.$$

Differentiate:

$$\frac{dL}{dp} = \alpha AC^{-\alpha} p^{\alpha-1} - \lambda Qe^{-\lambda p}.$$

At optimum:

$$\alpha AC^{-\alpha} p^{\alpha-1} = \lambda Qe^{-\lambda p}.$$

Taking logarithms:

$$\log(\alpha A) - \alpha \log C + (\alpha - 1) \log p = \log(\lambda Q) - \lambda p.$$

Rearrange:

$$\lambda p = \alpha \log C + \log \left(\frac{\lambda Q}{\alpha A} \right) - (\alpha - 1) \log p.$$

Ignoring the slowly varying $\log p$ term:

$$p^{\star} \approx \frac{\alpha}{\lambda} \log C + \text{constant}.$$

This explains why the optimal precision can increase slowly with compute.

Code Walkthrough

Symmetric Per-Tensor Quantization

```
import torch

def quantize_symmetric_per_tensor(w: torch.Tensor, bits: int):
    qmax = 2 ** (bits - 1) - 1
    max_abs = w.abs().max()
    scale = max_abs / qmax
    q = torch.round(w / scale).clamp(-qmax, qmax)
    w_hat = q * scale
    return w_hat, q, scale
```

This code implements a simple signed symmetric quantizer.

- `qmax` determines the largest representable integer.
- `max_abs` finds the dynamic range of the tensor.
- `scale` maps the largest magnitude value to the edge of the integer range.
- `round` maps each value to the nearest quantization bin.
- `clamp` prevents overflow.
- `w_hat` is the dequantized tensor used to simulate quantization.

This is useful for modeling loss degradation, but it does not by itself provide fast low-bit inference. For speed, values must usually be packed and consumed by specialized kernels.

Quantization-Aware Linear Layer

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class QuantAwareLinear(nn.Linear):
    def __init__(self, in_features, out_features, bits=8, bias=False):
        super().__init__(in_features, out_features, bias=bias)
        self.bits = bits

    def quantize_weight(self):
        qmax = 2 ** (self.bits - 1) - 1
        max_abs = self.weight.detach().abs().max()
        scale = max_abs / qmax
        q = torch.round(self.weight / scale).clamp(-qmax, qmax)
        return q * scale
```

```

def forward(self, x):
    w_hat = self.quantize_weight()
    return F.linear(x, w_hat, self.bias)

```

The forward pass computes:

$$Y = Xq(W)^T.$$

The important training behavior is:

- the model stores high-precision weights;
- the forward pass uses quantized weights;
- the optimizer updates the high-precision master weights;
- the model learns weights whose quantized versions are useful.

Straight-Through Estimator

Rounding has zero derivative almost everywhere, so quantization-aware training often uses a straight-through estimator.

```

class RoundSTE(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x):
        return torch.round(x)

    @staticmethod
    def backward(ctx, grad_output):
        return grad_output

```

The approximation is:

$$\frac{\partial \text{round}(x)}{\partial x} \approx 1.$$

This is not mathematically exact, but it allows gradients to flow through the quantizer.

Post-Training Quantization Experiment

```

def evaluate_ptq_sensitivity(checkpoints, bitwidths, validation_loader):
    results = []

    for ckpt in checkpoints:
        model = load_model(ckpt.path)
        full_loss = evaluate_loss(model, validation_loader)

```

```

for bits in bitwidths:
    q_model = copy_model(model)
    quantize_all_linear_weights(q_model, bits=bits)

    quant_loss = evaluate_loss(q_model, validation_loader)
    delta_loss = quant_loss - full_loss

    results.append({
        "N": ckpt.num_parameters,
        "D": ckpt.num_tokens_seen,
        "bits": bits,
        "full_loss": full_loss,
        "quant_loss": quant_loss,
        "delta_loss": delta_loss,
        "D_over_N": ckpt.num_tokens_seen / ckpt.num_parameters,
    })

return results

```

The key measured quantity is:

$$\Delta L = L_{\text{quant}} - L_{\text{full}}.$$

The lecture's result is that ΔL increases predictably with D/N .

Fitting the Power Law

```

import numpy as np
from sklearn.linear_model import LinearRegression

def fit_power_law(D_over_N, delta_loss):
    x = np.log(np.asarray(D_over_N)).reshape(-1, 1)
    y = np.log(np.asarray(delta_loss))

    reg = LinearRegression().fit(x, y)

    gamma = reg.coef_[0]
    log_a = reg.intercept_
    a = np.exp(log_a)

    return a, gamma

```

This fits:

$$\log \Delta L = \log a + \gamma \log(D/N).$$

The slope γ is the scaling exponent.

Effective Parameter Count Calculation

```
def effective_parameter_count(loss_quant, loss_inf, A, alpha):  
    numerator = A  
    denominator = loss_quant - loss_inf  
    return (numerator / denominator) ** (1.0 / alpha)
```

This computes:

$$N_{\text{eff}} = \left(\frac{A}{L_{\text{quant}} - L_{\infty}} \right)^{1/\alpha}.$$

This is useful because it maps a precision penalty into a scale penalty.

Performance Intuition

Why Weight Quantization Is Usually Easier

Weights are fixed after training, so they can be calibrated carefully. Quantization can use:

- per-tensor scales;
- per-channel scales;
- groupwise scales;
- GPTQ-style second-order corrections;
- AWQ-style activation-aware scaling.

Because weights do not change at inference time, their statistics are easier to estimate.

Why Activation Quantization Is Harder

Activations vary with the input. They may contain rare outlier channels. If a quantization scale is chosen based on an outlier, then most normal values occupy only a small part of the quantization range.

For activation tensor X , a single-scale quantizer must handle:

$$\max |X|.$$

If one channel has unusually large values, then the quantization step Δ becomes large, increasing error for all other channels.

This is one reason activations are often more sensitive than weights.

Why Very Low Precision Has a Cliff

Each additional bit doubles the number of representable states:

$$p \mapsto p + 1 \quad \Rightarrow \quad 2^p \mapsto 2^{p+1}.$$

Going from 8 bits to 7 bits changes states from 256 to 128. Going from 4 bits to 3 bits changes states from 16 to 8. The relative loss of representational granularity becomes severe at low bitwidth.

This explains why degradation can be mild at 8 bits but dramatic at 3 bits.

Why Overtraining Increases Sensitivity

Let θ be the trained parameter vector and ϵ be quantization noise:

$$\hat{\theta} = \theta + \epsilon.$$

A second-order Taylor expansion gives:

$$L(\theta + \epsilon) \approx L(\theta) + \nabla L(\theta)^T \epsilon + \frac{1}{2} \epsilon^T H(\theta) \epsilon.$$

Near a trained solution, the gradient term may be small, leaving:

$$\Delta L \approx \frac{1}{2} \epsilon^T H(\theta) \epsilon.$$

If overtraining moves the solution into a sharper region, then the Hessian $H(\theta)$ has larger curvature in important directions. The same quantization noise then causes larger loss.

This gives a mechanistic intuition for the empirical result:

$$D/N \uparrow \quad \Rightarrow \quad \Delta L_{\text{PTQ}} \uparrow.$$

Memory-Bound Versus Compute-Bound Implications

For inference, low precision is often valuable because memory bandwidth is a bottleneck. Weight-only quantization can reduce the amount of data loaded from memory.

For training, the story is more complicated. Training large transformers is often compute-bound in large matrix multiplications, so low precision is useful only if tensor cores can perform the actual matrix multiplications faster.

If low-bit values must be dequantized into FP16 before multiplication, then the operation may save memory but not achieve true low-precision compute throughput.

Common Misconceptions

Misconception 1: More Pre-Training Always Helps the Served Model

More pre-training usually helps the high-precision checkpoint, but it may hurt the quantized served model if quantization sensitivity grows faster than the full-precision loss improves.

The served loss is:

$$L_{\text{serve}} = L_{\text{full}} + \Delta L_{\text{PTQ}}.$$

The first term decreases with data, while the second can increase with D/N .

Misconception 2: Quantization Is Just Random Independent Noise

Quantization can be modeled as noise for intuition, but practical methods such as GPTQ and AWQ introduce structured, data-dependent, and layer-dependent effects. The lecture’s empirical point is that the broad scaling trends remain robust across several common methods.

Misconception 3: All Parts of the Model Have the Same Precision Sensitivity

Weights, activations, and KV cache do not behave the same way. Activations are often more sensitive than weights. KV cache can be tolerable at moderate precision but may degrade sharply at very low precision.

Misconception 4: 8-Bit and 4-Bit Are Just Linearly Different

Bitwidth is not linearly related to representational capacity. The number of states is exponential:

$$\text{states} = 2^p.$$

Thus, the damage from reducing precision is often strongly nonlinear.

Misconception 5: Hardware Speedups Automatically Translate to Training Wins

Even if hardware advertises higher low-precision throughput, practical training may not improve unless kernels, memory layout, scaling, accumulation, and numerical stability are all handled correctly.

Practical Takeaways

- Treat precision as a scaling variable, not merely an implementation detail.
- For post-training quantization, overtrained models are more fragile.
- The token-to-parameter ratio D/N is a key predictor of PTQ degradation.
- The best full-precision checkpoint may not be the best quantized checkpoint.
- Weight quantization is usually easier than activation quantization.
- KV cache quantization is crucial for long-context inference but can become fragile at very low bitwidth.
- Low precision provides hardware benefits through reduced memory traffic and higher tensor-core throughput, but only if kernels are truly optimized for the low-precision format.
- The trade-off between precision and model size is nonlinear.
- Effective parameter count is a useful way to interpret the loss penalty of quantization.
- Optimal precision can increase with compute budget, approximately like $p^* \propto \log C$ in the lecture's fitted framework.

Project or Experiment Ideas

Experiment 1: PTQ Degradation Versus Token-to-Parameter Ratio

Train several small transformers with the same architecture but different token budgets. Quantize each checkpoint to 8, 6, 4, and 3 bits. Plot:

$$\Delta L \text{ versus } D/N$$

on a log-log scale. Check whether the relationship is approximately linear.

Experiment 2: Weight Versus Activation Sensitivity

Implement three variants:

- weight-only quantization;
- activation-only quantization;
- weight and activation quantization together.

Compare validation loss at different bitwidths. This tests whether activations are more fragile than weights.

Experiment 3: KV Cache Quantization

During autoregressive decoding, quantize stored keys and values to different precisions. Measure:

- perplexity;
- generation quality;
- memory usage;
- tokens per second;
- degradation as sequence length increases.

This directly tests the long-context serving trade-off.

Experiment 4: Effective Parameter Count

Fit a high-precision scaling curve:

$$L_{\text{full}}(N) = L_{\infty} + AN^{-\alpha}.$$

Then quantize larger models and compute:

$$N_{\text{eff}} = \left(\frac{A}{L_{\text{quant}} - L_{\infty}} \right)^{1/\alpha}.$$

Plot:

$$\frac{N_{\text{eff}}}{N} \quad \text{versus} \quad p.$$

Experiment 5: Kernel-Level Benchmarking

Benchmark matrix multiplication in:

- FP32;
- BF16;
- FP16;
- FP8;
- simulated INT8;
- simulated INT4.

Measure:

- achieved TFLOP/s;
- memory bandwidth;
- arithmetic intensity;
- kernel launch overhead;
- dequantization overhead.

Compare theoretical speedup with realized speedup.

Final Mental Model

Low precision buys hardware efficiency by reducing memory movement and increasing potential tensor-core throughput. But it also injects perturbations into the model. The loss caused by those perturbations is not random chaos; it follows predictable scaling trends.

For post-training quantization, the key predictor is the token-to-parameter ratio:

$$\Delta L_{\text{PTQ}} \propto \left(\frac{D}{N} \right)^\gamma.$$

This means overtrained models are more sensitive to quantization.

For quantization during training, low precision can be interpreted as reducing effective parameter count:

$$N_{\text{eff}}(p) = r(p)N.$$

The function $r(p)$ is close to 1 at high precision but can collapse at very low precision, especially for activations and KV cache.

The overall lesson is:

Precision, model size, and data form a three-way scaling trade-off. Low precision is not free, but its cost is predictable enough to reason about scientifically and engineer around practically.